

```

#include <iostream>
using namespace std;
const int MAX = 100;
const int MAXE = MAX * MAX;

int parent[MAX]; // manual Union-Find structure

// find the "group leader" of a node (with path compression)
int findParent(int node) {
    if (parent[node] == node) return node;
    parent[node] = findParent(parent[node]);
    return parent[node];
}

// join two groups together
void unionNodes(int a, int b) {
    int pa = findParent(a);
    int pb = findParent(b);
    if (pa != pb) parent[pa] = pb;
}

struct Edge {
    int u, v, weight;
};

// simple manual sort (ascending by weight)
void sortEdges(Edge edges[], int e) {
    for (int i = 0; i < e - 1; i++) {
        for (int j = 0; j < e - i - 1; j++) {
            if (edges[j].weight > edges[j + 1].weight) {
                Edge temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

void inputEdges(Edge edges[], int n, int e) {
    cout << "Enter edges (Source Destination Weight):\n";
    for (int i = 0; i < e; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        if (u >= 1 && u <= n && v >= 1 && v <= n && u != v) {
            edges[i].u = u - 1;
            edges[i].v = v - 1;
            edges[i].weight = w;
        } else {
            cout << "Invalid edge. Try again.\n";
            i--;
        }
    }
}

```

```

// Kruskal's Algorithm - picks smallest edges that don't form a cycle
void kruskalMST(Edge edges[], int n, int e) {
    sortEdges(edges, e);
    for (int i = 0; i < n; i++) parent[i] = i; // each node its own leader
    int edgeCount = 0, totalWeight = 0;
    cout << "\nEdge \tWeight\n";
    for (int i = 0; i < e && edgeCount < n - 1; i++) {
        int u = edges[i].u, v = edges[i].v;
        if (findParent(u) != findParent(v)) { // no cycle formed
            unionNodes(u, v);
            cout << u+1 << " - " << v+1 << "\t" << edges[i].weight << "\n";
            totalWeight += edges[i].weight;
            edgeCount++;
        }
    }
    if (edgeCount < n - 1) {
        cout << "\nGraph is disconnected. MST incomplete.\n";
        cout << "Edges used: " << edgeCount << " / " << n - 1 << "\n";
    }
    cout << "\nTotal MST Weight = " << totalWeight << endl;
}

int main() {
    int n, e;
    Edge edges[MAXE];
    cout << "Enter number of nodes: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> e;
    inputEdges(edges, n, e);
    kruskalMST(edges, n, e);
    return 0;
}

```

WHEN TO USE KRUSKAL'S (Scenario Triggers)

- ▶ Question asks for Minimum Spanning Tree (MST) — same goal as Prim's, different method
- ▶ Graph is SPARSE (relatively few edges) or given directly as an EDGE LIST
- ▶ You need to process edges in strictly sorted (increasing weight) order
- ▶ Graph may NOT be fully connected initially — Kruskal's naturally works with edge lists from disjoint pieces
- ▶ Question hints at "union", "grouping", "merging components", or "avoid cycles"

Kruskal's vs Prim's — Which One?

- ▶ Graph is SPARSE / edges given as list → Kruskal's (sort + Union-Find)
- ▶ Graph is DENSE (near-complete) → Prim's is more efficient
- ▶ Kruskal's builds MULTIPLE tree fragments that merge together over time
- ▶ Prim's builds ONE single tree that grows outward from a start node
- ▶ Both always produce the same TOTAL MST weight for the same graph

MANUAL TRACE METHOD (Node-by-Node Edge List, from Class Notes)

Use this exact process — it matches the class notes' per-node edge listing format:

- ▶ 1. List ALL edges sorted by weight (ascending), OR group them by each node as shown in notes.
- ▶ 2. Go through edges in increasing weight order, one at a time.
- ▶ 3. For each edge (u,v): check if u and v are ALREADY in the same group (same 'root'/leader).
- ▶ 4. If NOT in the same group → SELECT this edge (box it), then UNION the two groups together.
- ▶ 5. If ALREADY in the same group → REJECT this edge (cross it out with X) — it would form a cycle.
- ▶ 6. Stop once you've selected exactly (n-1) edges, where n = total number of nodes.
- ▶ 7. Sum all selected (boxed) edge weights = Total MST weight.

COMMON GOTCHAS (Exam Traps)

⚠ Watch out for:

Sorting is MANDATORY — Kruskal's only works correctly if edges are processed in increasing weight order. Skipping this gives a wrong (non-minimum) tree.

Cycle check is the CORE logic — use Union-Find (findParent) to check 'same group' in $O(\text{near-1})$ time, not a slow manual cycle search.

Path compression matters — `parent[node] = findParent(parent[node])` speeds up future lookups; without it, worst-case behavior degrades badly on large graphs.

Disconnected graph — Kruskal's will simply select fewer than (n-1) edges and stop; you must explicitly report the graph is disconnected.

Equal-weight edges can be processed in ANY order among themselves — this may produce a different (but equally valid) MST than Prim's picks.

EXAMPLE 1 — Small Graph (Trace by Hand)

Graph (undirected, weighted): A-B=2, A-C=3, B-C=1, B-D=4, C-D=5

Sorted edges: B-C(1), A-B(2), A-C(3), B-D(4), C-D(5)

Step-by-step trace:

- ▶ B-C (1): B and C in different groups → SELECT. Union{B,C}.
- ▶ A-B (2): A and B in different groups → SELECT. Union{A,B,C}.
- ▶ A-C (3): A and C now in SAME group → REJECT (would form a cycle).
- ▶ B-D (4): B and D in different groups → SELECT. Union{A,B,C,D}.
- ▶ 3 edges selected = $n-1$ ($n=4$) → STOP, MST complete.

MST Edges: B-C(1), A-B(2), B-D(4) → Total Weight = 7

Note: Same total weight as Prim's example on the same graph — confirms both algorithms agree, even though the exact edges picked can differ.

EXAMPLE 2 — Disconnected Graph

Graph: A-B=5, C-D=3 (two separate components)

Sorted: C-D(3), A-B(5). Process both — SELECT C-D, SELECT A-B.

Only 2 edges selected, but $n=4$ needs $n-1=3$ edges. Exam expects you to say: "MST incomplete — the graph is disconnected. Only individual spanning trees exist for each component (A-B and C-D separately), no single MST connects all 4 nodes."

EXAMPLE 3 — Real-World Scenario (CP-style)

Scenario: "A telecom company has a list of possible fiber links between towns, each with an installation cost. They want to connect all towns at minimum total cost, and only a SPARSE set of possible links exists."

How to apply Kruskal's: since only a sparse/limited list of possible links exists (not all town pairs), Kruskal's is the natural fit — sort the given links by cost, use Union-Find to avoid forming loops (redundant connections), and stop once all towns are connected.

EXAMPLE 4 — Cycle Rejection Twist (Medium/Hard style)

Scenario: exam gives a graph and asks "which edges get REJECTED and why?" rather than just asking for the final MST.

Correct handling: for each rejected edge, you must explicitly state $\text{findParent}(u) == \text{findParent}(v)$ BEFORE that edge was considered — meaning a path already exists between u and v through previously selected edges, so adding this edge would create a redundant cycle. Class notes show this with an 'X' mark next to rejected edges — always justify the X, don't just mark it.

QUICK CHECKLIST BEFORE SUBMITTING AN ANSWER

Before you finalize any Kruskal's answer, check:

- Did I sort ALL edges by weight first, before selecting any?
- Did I correctly use Union-Find (not just 'looks connected') to check for cycles?
- Does my final selected-edge count equal $n-1$ (n = number of nodes)?
- If graph is disconnected, did I explicitly state MST is incomplete?
- For rejected edges, did I justify WHY (same group already) not just mark them rejected?